Chapter 2: Utilizing Tools From The Registry

Utilizing tools from the Registry

As we have already seen it should be possible to containerize almost any project. Since we are in between Dev and Ops, let's pretend that some developer has made a project that is ready for production with a README that instructs what to install and how to run the application. Now we as the container experts can containerize it in seconds.

Open the https://github.com/docker-hub-examples/python-pandas/blob/master/README.md# project, and read through the README and think about how to transform it into a Dockerfile. Thanks to the Registries we should be able to decipher what we will need to do even if we have no clue about the language or technology!

We are ready to clone the repository if, which you may have already done. After that is done, let's start with a Dockerfile. We know that we need to install Ruby and whatever dependencies it has. Let's place the Dockerfile in the project root.

```
FROM ruby:2.7.2-slim-buster
RUN ruby-2.7.0

EXPOSE 3000

WORKDIR /app/hello
```

Ok these are the basics, we have FROM a Ruby version, EXPOSE 3000 was told at the bottom of the README and WORKDIR /app/hello is the convention.

The next are told to us by the README. We won't need to copy anything from outside of the container to run these.

```
RUN gem install bundler:2.2.2

# Install the application specific gem dependencies to the container
COPY Gemfile* ./

RUN bundle install --jobs=3 --retry=3

# Copy the application to the container
COPY . /app/hello/.
```

Here we did a quick trick to separate installing dependencies from the part where we copy the source code in. The COPY will copy both files Gemfile and Gemfile.lock, and then bundle install will run. If we change only the dependency layer if we ever need to make changes to the source code. The same kind of caching trick works in many other languages or frameworks, such as Node.js.

And finally, we copy the project and follow the instructions in the README:

```
# Copy the application to the container
COPY . /app/hello/.

# Run this the production mode since we have no interest of developing in this image inside the container.
ENV RAILS_ENV=production

RUN rails db:migrate && rails db:seed

# Finally we can run the rails server.
RUN chmod +x ./run_server.sh

CMD ./run_server.sh -b 0.0.0.0
```

Ok, Let's see how well mimicking the README worked for us and run the following another that builds the image and then runs it with the port 3000 published:

```
docker build . -t main/project && docker run -p 3000:3000 main/project
```

After a while of waiting, the application starts in port 3000 in production mode... and you can access it in http://localhost:3000 unless you have a Mac with M1 or M2 processor.

Building the image with a more recent Mac

If you have a more recent Mac that has M1 or M2 ARM processor, building the image fails:

```
= DEBUG [7/8] RUN rails db:migrate RAILS_ENV=production
= [7/8] RUN rails db:migrate RAILS_ENV=production
/usr/local/bundle/gems/bootsnap-1.4.4/lib/bootsnap/load_path_cache.rb:102:in `require': cannot load such file -- bootsnap/bootsnap.bundle (LoadError)
```

This can be fixed by changing the following line in the the Gemfile.lock

```
x86_64-linux
```

to the form:

```
aarch64-linux
```

The reason for the problem is that the the Gemfile.lock that defines the exact version of the installed libraries (in Some ways is Ruby Project's generated with a Linux system that has an Intel processor. The Gem Nokogiri # has prebuilt binaries for the ARM processor, but the Bootsnap # does not. The latest version of the Gem is a more recent Mac, it is now just easiest to make a change in the Gemfile.lock.

Using prebuild Docker images

Instead of starting with a plain base image such as ubuntu, the Dockerfile in the example takes as starting point image: ruby:2.7. Ruby has a pre-build image hosted in Docker Hub. The Rust programming language Docker images, such as those for Ruby and Python, are Docker images that come with a specific version of Debian and the specified programming language version already configured. These images are provided by the Docker community and the language's maintainers and are available on Docker Hub.

One of the benefits of using a pre-build image is simplicity of setting up the development environment. You don't need to manually install and configure the language runtime and dependencies.

Exercise
1.9. Spring
Max. 90 min | Min. 30 min | Difficulty: 2

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

Create a Dockerfile for an old Java Spring project that can be found from the course repository #.

The setup should be straightforward with the README instructions. Tips to get you started:

There are many options for running Java, you may use the eclipse-temurin:8 #. (**FROM eclipse-temurin:8**) **-** The jar, (non-minified of minifying it manually. Pick the one by using the README and Docker Hub page.

You've completed the exercise when you see "Success! message in your browser.

Submit the Dockerfile and the commands you used to run the container.

```
FROM eclipse-temurin:8
WORKDIR /usr/src/app
COPY . .
RUN chmod +x mvnw
COPY -from=builder /app/target/spring-petclinic-2.7.3.jar*
docker build -t hello .
docker run --name frontend -d -p 3000:3000 hello
```

Word count: 25

The course project and mandatory exercises

The following three exercises will start a larger project that we will configure in chapters 3 and 4. They will require you to use everything you learned up until now. If you need to modify a Dockerfile in some later exercises, feel free to do it on top of the Dockerfiles you create now.

The course project related exercises are mandatory and you should do these to get a passing grade.

Exercise
1.10. Hello frontend (mandatory exercise)
Max. 90 min | Min. 30 min | Difficulty: 2

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

A good developer creates web written READMEs. Such that they can be used to create Dockerfiles with ease.

Clone, fork or download the project from https://github.com/docker-hy/material-applications/tree/main/example-frontend# and give a command so that the project runs in a Docker container with port (port exposed and published) so when you start the container and navigate to http://localhost:port you will see a message if you're successful.

note that the port given is reserved in the more recent OSX versions, so you have to use some other host port

As in other exercises, do not alter the code of the project

TIPS

- The project has install instructions in README
- Note that the app starts to accept connections when "Accepting connections at http://localhost:8080" is printed to the terminal. Port 8080 is the default port. You do not have to install anything non-suitable container.
- The project might not work with the newest Node.js versions.

Give the Dockerfile and the commands you used for building and running the image in the answer.

```
FROM node:18
WORKDIR /usr/src/app
COPY . .
RUN npm install && npm install -g serve
RUN npm run build
CMD["serve", "-s", "-l", "3000", "build"]
docker build -t frontend .&& docker run --name frontend -d -p 3000:3000 frontend
```

Word count: 26

Exercise
1.11. Hello backend (mandatory exercise)
Max. 90 min | Min. 30 min | Difficulty: 2

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

Clone, fork or download a project from https://github.com/docker-hy/material-applications/tree/main/example-backend# #.

Create a Dockerfile for the project (example backend). Start the container with port (8080 published).

When you start the container and navigate to http://localhost:8080/ping# if you should get a "pong" as a response.

Do not alter the code of the project

TIPS

- depending on your base image, you might need Ed #.
- If you have the file /bin/ping, you might need to build the image with an extra option: **docker build --platform linux/amd64 -t .** (**-platform** ...)

Submit the Dockerfile and the commands used as the answer.

```
FROM golang:1.19
WORKDIR /usr/src/app
COPY . .
RUN go build
RUN go test ./...
CMD ./server -p 8080
CMD[ "./server"]
docker build -t backend .&& docker run --name backend -d -p 8080:8080 backend
```

Word count: 34

Exercise
1.12. Environment (mandatory exercise)
Max. 90 min | Min. 30 min | Difficulty: 2

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

Start both the frontend and the backend with the current port-exposed and port-published of 8 in Dockerfiles with the necessary information from both README.frontend # and README.backend #.

Ignore the backend configuration until the frontend sends requests to BACKEND_URL/ping when you press the button.

You know that the configuration is ready when the button for 1.14 of frontend responds and turns green.

Do not alter the code of either project

To get the configurations right, it is important to understand the interplay of browser and the app.

- at first the browser gets HTML, file and the JavaScript from the frontend container
- the browser runs the JavaScript code. that renders the App and sets all the button listeners
- once the button for Exercise 1.14 is pressed, frontend does a request to the backend container

So the key thing is that the frontend code is executed by the browser, the frontend container does not carry code. It just serves the HTML, and the JavaScript code for the browser.

The following figure might help you with setting up the port mappings:

TIPS

- When configuring web applications keep the browser developer console at hand. It will show you the network requests and console logs. It is important for debugging configuring cross-origin requests in the backend (CORS of the backend project).
- The developer consoles has multiple views, the most important ones are Console and Network. Console shows the logs and Network shows the list of information or whole messages are being sent and which, successful, are important

Submit the edited Dockerfiles and commands used to run.

```
FROM node:18
WORKDIR /usr/src/app
COPY . .
RUN npm install && npm install -g serve
ENV REACT_APP_BACKEND_URL=http://localhost:8080/
RUN npm run build
CMD["serve", "-s", "-l", "3000", "build"]
docker build -t frontend .&& docker run --name frontend -d -p 3000:3000 frontend
FROM golang:1.19
WORKDIR /usr/src/app
COPY . .
RUN go build
RUN go test ./...
ENV PORT=8080
ENV FRONT_URL=http://localhost:3000/
CMD[ "./server"]
docker build -t backend .&& docker run --name backend -d -p 8080:8080 backend
```

Word count: 76

Publishing projects

Go to https://hub.docker.com/r/ to create an account. You can configure Docker hub to build your images for you, but using `docker` works as well.

Let's publish the pr-dbs image, that we did in a previous section. Log in and navigate to your repository # and press *Create Repository*. The namespace is your Docker Hub username, and the name is the repository name. For now, let's stick to personal accounts and write something descriptive such as *student-of-docker-2*. If repository name. We will need to remember it in chapter 3.

Set visibility to public.

And the last thing we need is to authenticate our push by logging in:

```
$ docker login
```

Next, you will need to rename the image to include your username, and then you can push it:

```
$ docker tag pr-dbs tonamurkamo/rikkaprikaap
$ docker push tonamurkamo/rikkaprikaap
```

Exercise
1.13. Homework
Max. 90 min | Min. 30 min | Difficulty: 2

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

Create Dockerfiles for an application or any other project of yours and publish this project. This can be any project, so long as the project is free (or, there are options of freemium) and open-source.

Make sure you have at least a basic description and instructions for how to run the application in the Repository overview of the project Docker Hub page.

For the answer provide the link to the project in Docker Hub in **the form username/repository**

Answer

tonamurkamo/pakerto_fullstack

Your answer was correct.

Exercise
1.14. Cloud deployment
Max. 90 min | Min. 30 min | Difficulty: 2

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

It is time to wrap up the project and run a containerized app in the cloud.

You can take any web app, eg. an example or version from this part or your own app, and deploy it to some cloud provider.

There are plenty of options, and most providers provide a free tier. Here are some options to consider:

- Heroku (only #! Bad documentation, you need likely need google)
- Render.com # (has a free student plan through GitHub Student Developer Pack offer)
- Fly.io # (easy to use but unfortunately no longer has a free tier)

If you know a good cloud service for the purposes of this exercise, please let us know, we know about Amazon AWS, Google Cloud and Azure already.

Submit the Dockerfile, a brief description of what you did, and a link to the running app.

Dockerfile
FROM node:18
WORKDIR /usr/src/app
COPY --chown=node:node .
RUN rm dist/static/fonts/*woff2 dist/*.zip
USER node
EXPOSE 3000
CMD npm run start

Description of the steps to deploy
First the following image was created and pushed to docker hub:
tonamurkamo/pakerto_fullstack:latest
Then a new app was created on Fly.io using the following commands:

Deployment of the app to Fly.io

docker tag my-patientor thomastoumasu/patientor_fullstack:linux-amd64
docker push thomastoumasu/patientor_fullstack:linux-amd64

Then this image was deployed on render as:
New -> Web Service -> Existing Image -> Image URL:
thomastoumasu/patientor_fullstack:linux-amd64
with setting PORT 3001 in Render -> Environment

Link to the running app:
https://patientor-fullstack-linux-amd64.onrender.com

Word count: 84

↑ Chapter front page

You've reached the end of this topic.

Proceed to the next topic

Next page:
Summary

About

The University of Helsinki MOOC Center makes high-quality online education possible by developing and researching educational software and online learning materials. Teachers both within and without the University of Helsinki rely on our tools to create impactful teaching materials. Our popular Massive Open Online Courses (MOOCs) have been available through MOOC.fi since 2012.

This website is powered by an open source software developed by the University of Helsinki MOOC Center. Star the project on GitHub: Project Github.

Privacy

Accessibility statement

MOOC.FI HELSINGIN YLIOPISTO HELSINGFORS UNIVERSITET UNIVERSITY OF HELSINKI

GIVE FEEDBACK